

SISTEMAS OPERATIVOS

Práctica 1**Procesos, llamadas al sistema e IPC en Linux**

Ingeniería de Sistemas — Sexto semestre

Propósito de la práctica. Comprender experimentalmente cómo Linux crea y administra procesos y cómo procesos independientes se comunican utilizando mecanismos IPC proporcionados por el sistema operativo. El conjunto de datos se utiliza únicamente como carga de trabajo para observar estos mecanismos; la práctica no evalúa estructuras de datos avanzadas ni técnicas de indexación.

1. Objetivos de aprendizaje

- Distinguir un programa de un proceso en ejecución e identificar PID, PPID, estado y recursos asociados.
- Utilizar llamadas al sistema y mecanismos POSIX para acceder a archivos y realizar comunicación entre procesos.
- Implementar comunicación entre dos procesos no emparentados mediante tuberías nombradas (FIFO).
- Implementar una segunda versión de la comunicación mediante memoria compartida y un mecanismo explícito de sincronización.
- Observar el comportamiento de los procesos y sus llamadas al sistema utilizando herramientas de Linux.
- Analizar situaciones de bloqueo, terminación inesperada y liberación de recursos del sistema operativo.

2. Descripción general del problema

Cada grupo trabajará con un dataset en formato CSV. El programa deberá permitir realizar consultas sencillas sobre los registros del archivo. No se exige implementar tablas hash, árboles ni otros índices. La búsqueda puede realizarse secuencialmente sobre el archivo.

La solución estará dividida en dos programas independientes:

- **Cliente:** presenta una interfaz de línea de comandos, recibe los criterios de consulta del usuario y muestra las respuestas.
- **Servidor:** abre el archivo CSV, ejecuta las búsquedas solicitadas y devuelve los resultados al cliente.

Cliente y servidor deben ejecutarse como procesos separados y no emparentados; por ejemplo, iniciados desde dos terminales distintas.

3. Arquitectura mínima

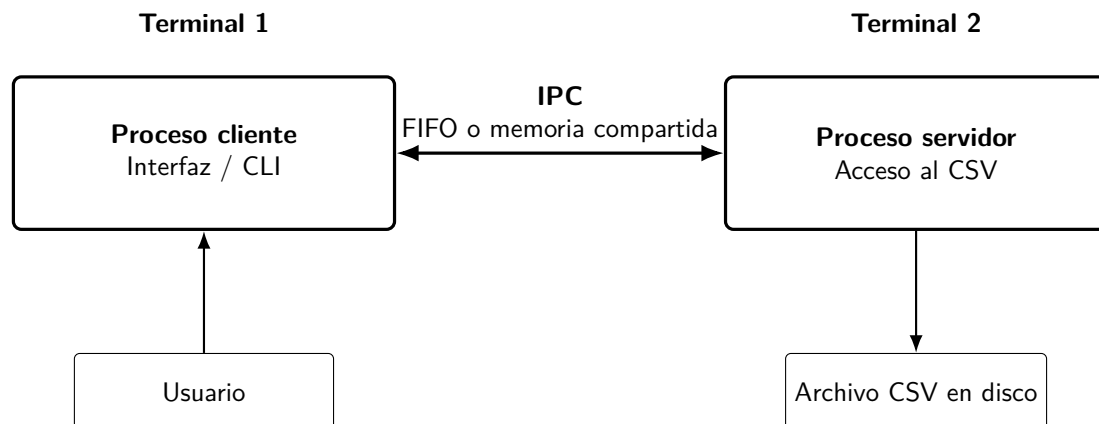


Figura 1: Arquitectura mínima cliente-servidor.

La arquitectura deberá implementarse dos veces, manteniendo la misma funcionalidad visible para el usuario: una versión con FIFO y otra con memoria compartida.

4. Funcionalidad del programa

El menú debe adaptarse a las características del dataset seleccionado. Como mínimo deberá permitir:

1. Definir al menos dos criterios de búsqueda relevantes para el dataset.
2. Enviar una solicitud de consulta desde el cliente al servidor.
3. Mostrar en el cliente uno o varios registros coincidentes.
4. Informar explícitamente cuando no se encuentren resultados.
5. Finalizar de manera controlada ambos procesos.

No se evaluará la complejidad algorítmica de la búsqueda. El énfasis estará en el uso correcto de procesos, IPC, llamadas al sistema, sincronización y manejo de recursos.

5. Parte A — Comunicación mediante FIFO

Implemente la primera versión utilizando tuberías nombradas. Cliente y servidor deben poder iniciarse de manera independiente.

5.1 Requisitos mínimos

- Crear y utilizar las FIFO necesarias para enviar solicitudes y recibir respuestas.
- Realizar la comunicación usando operaciones de apertura, lectura, escritura y cierre.
- Definir un protocolo sencillo para distinguir una consulta, sus parámetros, la respuesta y la orden de terminación.

- Evitar escrituras parciales o mensajes ambiguos; documentar el formato utilizado.
- Manejar errores de apertura, lectura, escritura y ausencia del otro proceso.
- Eliminar o reutilizar correctamente los recursos creados para la comunicación.

5.2 Aspectos que debe poder explicar el grupo

- Qué ocurre si el cliente abre una FIFO antes de que exista el servidor.
- En qué momento puede bloquearse una operación `read()` o `write()`.
- Qué descriptores de archivo mantiene abiertos cada proceso.
- Qué recursos del sistema operativo intervienen durante la comunicación.

6. Parte B — Comunicación mediante memoria compartida

Implemente una segunda versión reemplazando las FIFO por memoria compartida. La funcionalidad para el usuario debe permanecer esencialmente igual.

6.1 Requisitos mínimos

- Crear una región de memoria compartida accesible por los dos procesos independientes.
- Definir dentro de esa región una estructura para almacenar la solicitud, los parámetros, la respuesta y el estado de la comunicación.
- Utilizar un mecanismo de sincronización adecuado para impedir condiciones de carrera y evitar espera activa. Se recomienda utilizar semáforos POSIX o equivalentes.
- Garantizar que el cliente no lea una respuesta antes de que el servidor haya terminado de escribirla.
- Liberar correctamente memoria compartida y recursos de sincronización al finalizar.

6.2 Aspectos que debe poder explicar el grupo

- Por qué la memoria compartida requiere sincronización explícita.
- Qué condición de carrera aparecería si cliente y servidor accedieran simultáneamente a la misma región.
- Qué diferencias existen entre pasar datos mediante FIFO y compartir directamente una región de memoria.
- Qué ocurre con los recursos IPC cuando un proceso termina de manera inesperada.

7. Acceso al dataset y uso de memoria

- El archivo CSV completo debe permanecer en disco. No se permite cargar todo el dataset en memoria para realizar las consultas.

- El servidor puede leer secuencialmente el archivo en cada consulta o mantener información auxiliar pequeña que facilite el procesamiento, siempre que esta no constituya una copia completa del dataset.
- Debe utilizarse memoria dinámica cuando sea necesaria y todo bloque reservado deberá liberarse correctamente.
- La solución debe funcionar correctamente con archivos suficientemente grandes para que sea observable la diferencia entre datos en disco y datos residentes en memoria.

8. Observación del sistema operativo

Además de ejecutar el programa, el grupo deberá observar su comportamiento desde Linux. Durante el desarrollo y la sustentación deberán utilizar, como mínimo, varias de las siguientes herramientas:

Herramienta	Uso esperado
<code>ps / pstree</code>	Identificar PID, PPID, estado y relación entre procesos.
<code>/proc/<pid>/</code>	Examinar información que el kernel expone sobre el proceso.
<code>lsdf</code>	Identificar archivos y descriptores abiertos.
<code>strace</code>	Observar llamadas al sistema realizadas por cliente y servidor.
<code>ipcs</code> o herramientas equivalentes	Inspeccionar recursos IPC cuando aplique.
<code>top / htop</code>	Observar estado y consumo de recursos durante la ejecución.

El archivo LEEME debe incluir al menos dos observaciones concretas obtenidas con estas herramientas. Por ejemplo: una secuencia de llamadas al sistema observada con `strace` o el estado de un proceso mientras permanece bloqueado esperando una operación IPC.

Restricciones técnicas

- Lenguaje: C en Linux.
- Compilación mediante `gcc` y `Makefile`.
- Cliente y servidor deben compilarse como ejecutables separados.
- Las llamadas al sistema y funciones de biblioteca utilizadas deben comprobar sus valores de retorno cuando corresponda.

11. Entrega

La entrega deberá contener una carpeta con el nombre definido para el grupo e incluir:

1. Código fuente completo del cliente y del servidor.
2. Implementación de la versión FIFO.
3. Implementación de la versión con memoria compartida y sincronización.
4. `Makefile` que compile todos los ejecutables necesarios.

5. Archivo LEEME con: descripción del dataset; instrucciones de compilación y ejecución; protocolo de comunicación; recursos IPC utilizados; manejo de errores; y observaciones realizadas con herramientas de Linux.
6. Repositorio Git con el historial de desarrollo del grupo.

Sustentación

La práctica se revisará presencialmente. Se podrá seleccionar a cualquier integrante del grupo para ejecutar, explicar o modificar la solución. Durante la revisión se podrá solicitar:

- Identificar los procesos en ejecución y mostrar sus PID y PPID.
- Explicar una llamada al sistema observada con **strace**.
- Mostrar cuándo y por qué uno de los procesos queda bloqueado.
- Modificar un parámetro o una condición del protocolo de comunicación.
- Provocar la terminación de uno de los procesos y explicar el efecto sobre el otro.
- Explicar cómo se evita una condición de carrera en la versión con memoria compartida.
- Identificar y liberar un recurso IPC dejado por una ejecución incorrecta.

Evaluación

Aspecto	Porcentaje
Arquitectura cliente-servidor y manejo correcto de procesos	10 %
Comunicación mediante FIFO	10 %
Memoria compartida y sincronización	10 %
Manejo de errores, bloqueo y liberación de recursos	10 %
Uso y comprensión de llamadas al sistema y herramientas de observación	10 %
Funcionamiento general y calidad básica del código	10 %
Sustentación individual	40 %

Criterio fundamental: una solución que produzca la salida correcta pero cuyo funcionamiento, llamadas al sistema, mecanismos IPC o sincronización no puedan ser explicados por el estudiante podrá considerarse insuficiente.

Uso de herramientas de IA

Se mantienen las reglas generales del curso para el uso de herramientas de IA: pueden utilizarse como apoyo para consultar conceptos, depurar, analizar errores o estudiar alternativas de diseño. Cada integrante debe comprender y poder modificar todo el código que entrega. El uso significativo de estas herramientas debe declararse brevemente en el archivo LEEME.